Basic PHP

Philosphy of PHP

- You are a responsible and intelligent programmer
- You know what you want to do
- Some flexibility in syntax is OK - style choices are OK
- Lets make this as convienent as possible
- Sometimes errors fail silently

PHP examples

```
<h1>Hello from Cindy Liu's HTML Page</h1>
<p>
<?php
echo "Hi there. <br/>";
$answer = 6 * 7;
echo "The answer is $answer, what ";
echo "was the question again? <br/>";
?>
</p>
<p>Yes another paragraph.</p>
```

PHP examples

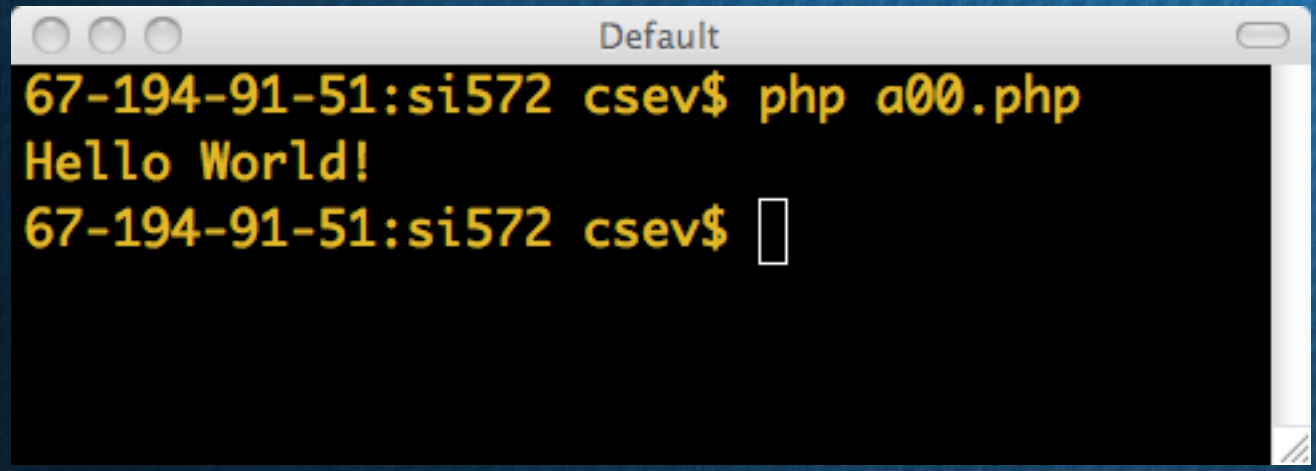
```
<h1>Hello from Cindy Liu's HTML Page</h1>
<p>
<?php
echo "Hi there. <br/>";
$answer = 6 * 7;
echo "The answer is $answer, what ";
echo "was the question again? <br/>";
?>
</p>
<p>Yes another paragraph.</p>
```

PHP examples

- You can run PHP from the command line - the output simply comes out on the Terminal

- It does not have to be part of a request-response cycle

```
<?php
echo("Hello World!");
echo("\n");
?>
```



Key Words

abstract and array() as break case catch class clone const continue declare default do else elseif end declare endfor endforeach endif endswitch endwhile extends final for foreach function global goto if implements interface instanceof namespace new or private protected public static switch $this throw try use var while xor

http://php.net/manual/en/reserved.php

Variable Names

- Start with a dollar sign ($) followed by a letter or underscore, followed by any number of letters, numbers, or underscores
- Case matters

```
$abc = 12;
$total = 0;
$largest_so_far = 0;
```

```
$abc = 12;
$total = 0;
$largest_so_far = 0;
```

http://php.net/manual/en/language.variables.basics.php

Variable Names

```php
<?php
$var = 'Bob';
$Var = 'Joe';
echo "$var, $Var";          // outputs "Bob, Joe"

$4site = 'not yet';         // invalid; starts with a number
$_4site = 'not yet';        // valid; starts with an underscore

?>
```

Variable Names Weirdness

- Things that look like variables but are missing a dollar sign can be confusing

```
$x = 2;
$y = x + 5;
print $y;
```

```
5
```

```
$x = 2;
y = $x + 5;
print $x;
```

```
Parse error
```

Expressions

- Completely normal like other languages (+ - / *)
- More agressive implicit type conversion

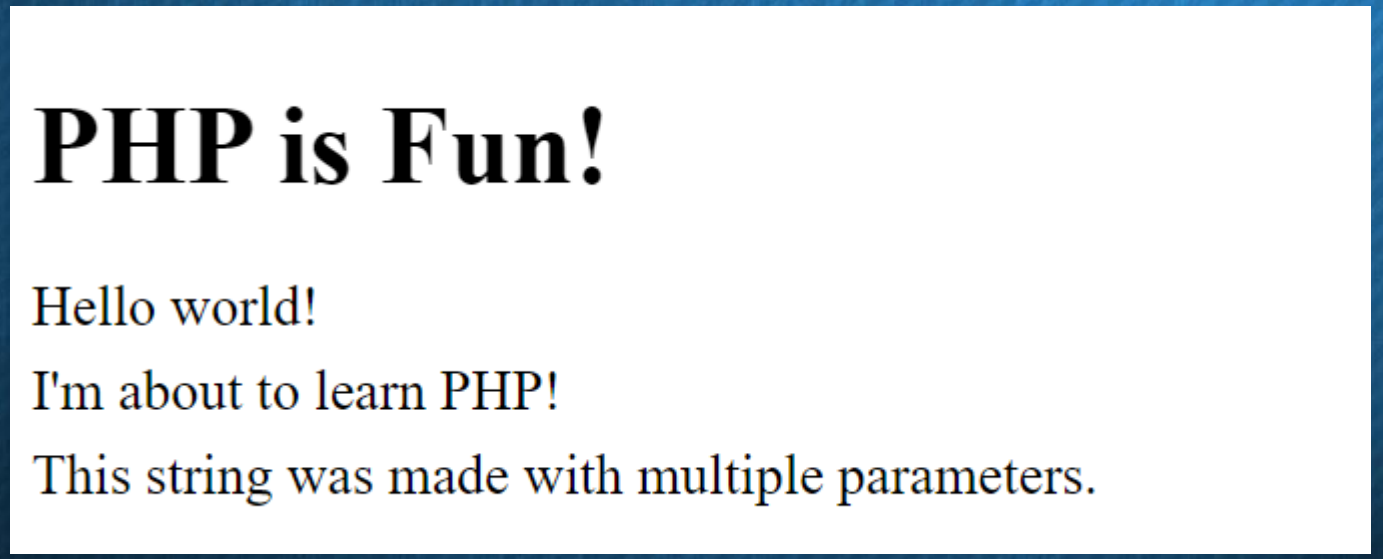
```
<?php
$x = "15" + 27;
echo($x);
echo("\n");
?>
```

Output

- With PHP, there are two basic ways to get output: echo and print.
- echo and print are more or less the same. They are both used to output data to the screen.
- The differences are small: echo has no return value while print has a return value of 1 so it can be used in expressions. echo can take multiple parameters (although such usage is rare) while print can take one argument. echo is marginally faster than print.

Output examples

```
<?php
echo "<h2>PHP is Fun!</h2>";
echo "Hello world!<br>";
echo "I'm about to learn PHP!<br>";
echo "This ", "string ", "was ", "made ", "with
multiple parameters.";
?>
```



Output examples

```
<?php
$txt1 = "Learn PHP";
$txt2 = "W3Schools.com";
$x = 5;
$y = 4;

print "<h2>" . $txt1 . "</h2>";
print "Study PHP at " . $txt2 . "
print $x + $y;
?>
```

Learn PHP

Study PHP at W3Schools.com

9

White space does not matter

```
<?php
$ans = 42;
if ( $ans == 42 ) {
print "Hello world!<br/>";
} else {
print "Wrong answer<br/>";
}
?>
```

```
<?php $ans = 42; if ( $ans == 42 ) { print
"Hello world!<br/>"; } else { print "Wrong answer<br/>"; }
?>
```

What style do you prefer

```php
<?php
$ans = 42;
if ( $ans == 42 ) {
print "Hello world!<br/>";
} else {
print "Wrong answer<br/>";
}
?>
```

```php
<?php
$ans = 42;
if ( $ans == 42 )
{
print "Hello world!<br/>";
}
else
{
print "Wrong answer<br/>";
}
?>
```

PHP Operators - Arithmetic Operators

Operator	Description	Example	Result
+	Addition	x=2 x+2	4
-	Subtraction	x=2 5-x	3
*	Multiplication	x=4 x*5	20
/	Division	15/5 5/2	3 2.5
%	Modulus (division remainder)	5%2 10%8 10%2	1 2 0
++	Increment	x=5 x++	x=6
--	Decrement	x=5 x--	x=4

PHP Operators - Assignment Operators

Operator	Example	Is The Same As
=	x=y	x=y
+=	x+=y	x=x+y
-=	x-=y	x=x-y
*=	x*=y	x=x*y
/=	x/=y	x=x/y
.=	x.=y	x=x.y
%=	x%=y	x=x%y

PHP Operators - Comparison Operators

Operator	Description	Example
==	is equal to	5==8 returns false
!=	is not equal	5!=8 returns true
>	is greater than	5>8 returns false
<	is less than	5<8 returns true
>=	is greater than or equal to	5>=8 returns false
<=	is less than or equal to	5<=8 returns true

PHP Operators – Logical Operators

Operator	Description	Example
&&	and	x=6 y=3 (x < 10 && y > 1) returns true
||	or	x=6 y=3 (x==5 || y==5) returns false
!	not	x=6 y=3 !(x==y) returns true

Arrays

- An array is a special variable, which can hold more than one value at a time.
- In PHP, there are three types of arrays: Indexed arrays - Arrays with a numeric index Associative arrays - Arrays with named keys Multidimensional arrays - Arrays containing one or more arrays

Indexed Arrays

- An indexed array stores each element with a numeric ID key.
- There are different ways to create an indexed array.
- In this example the ID key is automatically assigned:

```
$names  =  array("Peter","Mary","Joe");
```

- In this example the ID key is manually assigned:

```
$names[0]   =  "Peter";
$names[1]   =  "Mary";
$names[2]   =  "Joe";
```

Indexed Arrays

- The ID keys can be used in a script:

```
<?php
$names[0] = "Peter";
$names[1] = "Mary";
$names[2] = "Joe";
echo $names[1] . " and " . $names[2] .
" are ". $names[0] . "'s neighbours";
?>
```

- The code above will output:

Mary and Joe are Peter's neighbours

Associative Arrays

- An associative array, each ID key is associated with a value.
- When storing data about specific named values, an indexed array is not always the best way to do it.
- With associative arrays we can use the values as keys and assign values to them.
- In this example we use an array to assign ages to the different persons:

```
$ages = array("Peter"=>32, "Mary"=>30,
"Joe"=>34);
```

Associative Arrays

- This example is the same as example 1, but shows a different way of creating the array:

```
$ages['Peter']  =  "32";
$ages[‘Mary']  =  "30";
$ages['Joe']   =  "34";
```

Associative Arrays

- The ID keys can be used in a script:

```
<?php
$ages['Peter']  =  "32";
$ages[‘Mary']  =  "30";
$ages['Joe']  =  "34";
Echo "Peter is “ . $ages['Peter'] . “ years  old.";
?>
```

- The code above will output:
- Peter is 32 years old.

Multidimensional Arrays

- In a multidimensional array, each element in the main array can also be an array. And each element in the sub-array can be an array, and so on.
- In the next example we create a multidimensional array, with automatically assigned ID keys:

Multidimensional Arrays

```
$families  =  array
(
"Griffin"=>array
(
"Peter",
"Lois",
"Megan"
),
"Quagmire"=>array
(
"Glenn"
),
"Brown"=>array
(
"Clive",
"Loretta",
"Junior"
)
);
```

Multidimensional Arrays

- The array last page would look like this if written to the output:

```
Array
(
[Griffin]  =>  Array
(
[0]  =>  Peter
[1]  =>  Lois
[2]  =>  Megan
)
[Quagmire]  =>  Array
(
[0]  =>  Glenn
)
[Brown]  =>  Array
(
[0]  =>  Clive
[1]  =>  Loretta
[2]  =>  Junior
)
```

Multidimensional Arrays

- Lets try displaying a single value from the array above:

```
echo "Is " . $families['Griffin'][2] .
" a part of the Griffin family?";
```

- The code above will output:

Is Megan a part of the Griffin family?

Variable Name Weirdness

- Things that look like variables but are missing a dollar sign as an array index are unpredictable...:

```
$x = 5;

$y = Array("x" => "Hello");

print $y[x];
```

Hello

Strings

- String literals can use single quotes or double quotes
- The backslash (\) is used as an "escape" character in single quotes string
- Strings can span multiple lines - the newline is part of the string
- In double-quoted strings variable values are expanded
- The main difference between double quotes and single quotes is that by using double quotes, you can include variables directly within the string. It interprets the Escape sequences. Each variable will be replaced by its value.

Single Quote

```
<?php>
echo 'this is a simple string';
echo 'You can also have embedded newlines in strings this way
as it is okay to do';
// Outputs: Arnold once said: "I'll be back"
echo 'Arnold once said: "I\'ll be back"';
// Outputs: This will not expand: \n a newline
echo 'This will not expand: \n a newline';
// Outputs: Variables do not $expand $either
echo 'Variables do not $expand $either';
?>
```

Double Quote

```
<?php
Echo "this is a simple string";
Echo "You can also have embedded newlines in strings this way
as it is okay to do";
// Outputs: This will expand:
// a newline
Echo "This will expand: \n a newline";
// Outputs: Variables do 12
$expand = 12;
Echo "Variables do $expand\n";
?>
```

Comments

```
<?php
echo 'This is a test';
// This is a c++ style comment
/* This is a multi line comment
yet another line of comment */
echo 'This is yet another test';
echo 'One Final Test';
# This is a shell-style comment
?>
```

Strings

- Those “\n” and others in Single Quote and Double Quote are designed to use in output plain strings

- In HTML, you should use “nl2br”, or just use “
”

Expressions

- Expressions evaluate to a value. The value can be a string, number, boolean, etc...

- Expressions often use operations and function calls, and there is an order of evaluation when there is more than one operator in an expression

- Expressions can also produce objects like arrays

Expressions

High

Operator(s)	Type
( )	Parentheses
++ --	Increment/Decrement
!	Logical
* / %	Arithmetic
+ - .	Arithmetic and String
<< >>	Bitwise
< <= > >= <>	Comparison
== != === !==	Comparison
&	Bitwise (and references)
^	Bitwise
|	Bitwise

Operator Precedence

&&	Logical
||	Logical
? :	Ternary
= += -= *= /= .= %= &= != ^= <<= >>=	Assignment
and	Logical
xor	Logical
or	Logical

Low

Operators of Note

- Increment / Decrement (++ --)
- String concatenation (.)
- Equality (== !=)
- Identity (=== !==)
- Ternary (? :)
- Side-effect Assignment (+= -= .= etc.)
- Ignore the rarely-used bitwise operators (>> << ^ | &)

Increment / Decrement

- These operators allow you to both retrieve and increment / decrement a variable
- They are generally avoided in civilized code.

```
$x = 12;
$y = 15 + $x++;                    x is 13 and y is 27
echo "x is $x and y is $y \n";
```

Increment / Decrement

- Instead, we could use the code below to serve the same purpose

```
$x = 12;
$y = 15 + $x;                    x is 13 and y is 27
$x = $x + 1;
echo "x is $x and y is $y \n";
```

String Concatenation

- PHP uses the period character for concatenation because the plus character would instruct PHP to the best it could do to add the two things together, converting if necessary

```
$a = 'Hello ' . 'World!';
echo $a . "\n";                    Hello World!

$a = '12' . '13';
echo $a . "\n";                    1213
```

Equality(==) versus Identity(===)

- The equality operator (==) in PHP is far more aggressive than in most other languages when it comes to data conversion during expression evaluation.

```
if ( 123 == "123" ) print ("Equality 1\n");
if ( 123 == "100"+23 ) print ("Equality 2\n");
if ( FALSE == "0" ) print ("Equality 3\n");
if ( (5 < 6) == "2"-"1" ) print ("Equality 4\n");
if ( (5 < 6) === TRUE ) print ("Equality 5\n");
```

Equality(==) versus Identity(===)

```
// "===" means that they are identical
// "==" means that they are equal
// "!=" means that they aren't equal.

          false   null    array()  0       "0"     0x0     "0x0"   "000"   "0000"
false     ===     ==      ==       ==       ==     ==      !=      !=      !=
null      ==      ===     ==       ==       !=     ==      !=      !=      !=
array()   ==      ==      ===      !=       !=     !=      !=      !=      !=
0         ==      ==      !=       ===      ==     ===     ==      ==      ==
"0"       ==      !=      !=       ==       ===    ==      ==      ==      ==
0x0       ==      ==      !=       ===      ==     ===     ==      ==      ==
"0x0"     !=      !=      !=       ==       ==     ==      ===     ==      ==
"000"     !=      !=      !=       ==       ==     ==      ==      ===     ==
"0000"    !=      !=      !=       ==       ==     ==      ==      ==      ===
```